

ADR-001 — Choix de la stack technique

MarketHub

Date

2026 — Jour 1

Statut

Accepté

Décideurs

- Alphonse — Chef de projet / Backend
- Elisabeth — Développeur Frontend
- Dovane — Développeur Full-stack

Contexte

Développer une marketplace complète en 7 jours avec une équipe de 3 développeurs, en respectant les contraintes pédagogiques B2.

Décisions

Backend — Symfony PHP

- ORM Doctrine natif et mature
- API Platform génère OpenAPI automatiquement
- Maîtrise de l'équipe sur PHP
- Authentification JWT intégrée

Frontend — Vue.js + TypeScript

- Composition API simple et lisible
- Pinia pour le state management
- Tailwind CSS pour le responsive

- TypeScript obligatoire selon l'énoncé

Base de données — MySQL

- Full-text search natif
- Compatible Doctrine ORM
- Familiarité de l'équipe

Messagerie — WebSocket Ratchet

- S'intègre dans l'écosystème PHP
- Pas de serveur séparé nécessaire
- Gestion des connexions en temps réel

Authentification — JWT + Refresh Token

- Stateless — pas de session serveur
- Access token 15 minutes
- Refresh token 7 jours
- Sécurisé et standard REST

CI/CD — GitHub Actions

- Natif GitHub — gratuit
- Pipeline lint + test + build + deploy
- Déclenchement automatique sur push

Déploiement — Docker Compose

- Un seul fichier pour toute la stack
- App + BDD + Nginx reverse proxy
- Portable et reproductible

Alternatives rejetées

Alternative	Raison du rejet
FastAPI (Python)	Courbe d'apprentissage plus longue
React	Redux plus complexe que Pinia

PostgreSQL	MySQL suffisant pour le projet
Node.js	Équipe moins à l'aise

Architecture retenue

Présentation (Vue.js) ↓ Métier (Services Symfony) ↓ Données (Doctrine + MySQL)

Chaque couche ne connaît que celle du dessous. Respect strict des principes SOLID.

Conséquences

- Architecture en 3 couches strictes
- Séparation claire frontend/backend
- API REST documentée automatiquement
- Tests unitaires PHPUnit + Cypress e2e